

CS 532: 3D Computer Vision

Lecture 2

Enrique Dunn

edunn@stevens.edu

Gateway South 323



Programming and Image Processing

Philippos Mordohai & Matt Burlick

First programs

Go to colab.research.google.com

Add (+) new code cell

We will use python today

And throughout...

For loops

```
# array sum
import numpy as np
a = np.array([3, -5, 8, 12, 56, -17, 14])
total = 0
for i in range(0, a.shape[0]):
    total = total + a[i]
print('The sum is ' + str(total))
```

Code can be
pasted directly into
colab.

Tweak the start and end
point of range() - the end
point is NOT included in the
loop

Conditionals

```
# absolute value
```

```
x = -5.6
```

```
if (x < 0):
```

```
    x = -x
```

```
print('x: ' + str(x))
```

Conditionals - Part 2

```
perc = 0.91
if perc > 0.95:
    print('A')
elif perc > 0.90:
    print('A-')
elif perc > 0.70:
    print('Pass')
else:
    print('Aargh!')
```

Conditionals - Part 2, part b

```
perc = 0.8
if perc > 0.00:
    print('Aargh!')
elif perc > 0.70:
    print('Pass')
elif perc > 0.90:
    print('A-')
else:
    print('A')
```


Digital Images

- Images are *arrays* in python/numpy
 - The number of pixels in a row corresponds to width
 - The number of pixels in a column corresponds to height
- Most images contain either 1 (gray) or 3 color channels (RGB)
 - The value of each channel corresponds to the intensity of the channel
 - We will work with 8-bit images where the intensity ranges from 0 to 255 ($2^8 = 256$)

Loading and Displaying an Image

```
!pip install -q mediapy
import mediapy as media
import numpy as np
```

```
image_rgb = media.read_image('https://github.com/hhoppe/data/raw/main/image.png')
print(image_rgb.shape, image_rgb.dtype)  # It is a numpy array.
media.show_image(image_rgb)
```

```
image_gray = \
media.read_image('https://drive.google.com/uc?export=download&\
confirm=yes&id=1U-VRah1J45ep60J4EEyB_hBB1fu_DW1g')
print(image_gray.shape, image_gray.dtype)  # It is a numpy array.
media.show_image(image_gray)
```

Sub-images

```
image_gray = \
media.read_image('https://drive.google.com/uc?export=download&\
confirm=yes&id=1U-VRah1J45ep60J4EEyB_hBB1fu_DW1g')
```

```
top_right = image_gray[0:200, 312:512]
media.show_image(top_right)
```

```
square = np.array(image_gray[:, :])
```

```
square[20:140, 20:140]=np.zeros((120, 120), dtype='uint8')
media.show_image(square)
```

TODO:

1. Display 200x200 square at bottom left of the image
2. Display 300x300 square at the center of the image

Operating on All Pixels

```
[height, width] = image_gray.shape
```

```
for i in range(0, height):  
    for j in range(0, width):
```

Binary Image

```
[height, width] = image_gray.shape
```

```
output = np.zeros(image_gray.shape, dtype='uint8')
```

```
for i in range(0, height):  
    for j in range(0, width):  
        if (image_gray[i, j] > 128):  
            output[i, j] = 255
```

```
media.show_image(output)
```

Why no *else*?

TODO: make images
with more or fewer
white pixels

TODO: Conditionals on Brightness

What can we do with RGB images?

TODO: load an image of your choice and modify at least one of the color channels as a function of at least two of the channels

TODO: Negative Image

Image Filtering

- Apply to every pixel of the image the same operation
 - Noise reduction
 - Smoothing
 - Sharpening
- Operation represented by a window/filter/mask of weights to be applied to the **neighborhood** of each pixel

Image Filtering

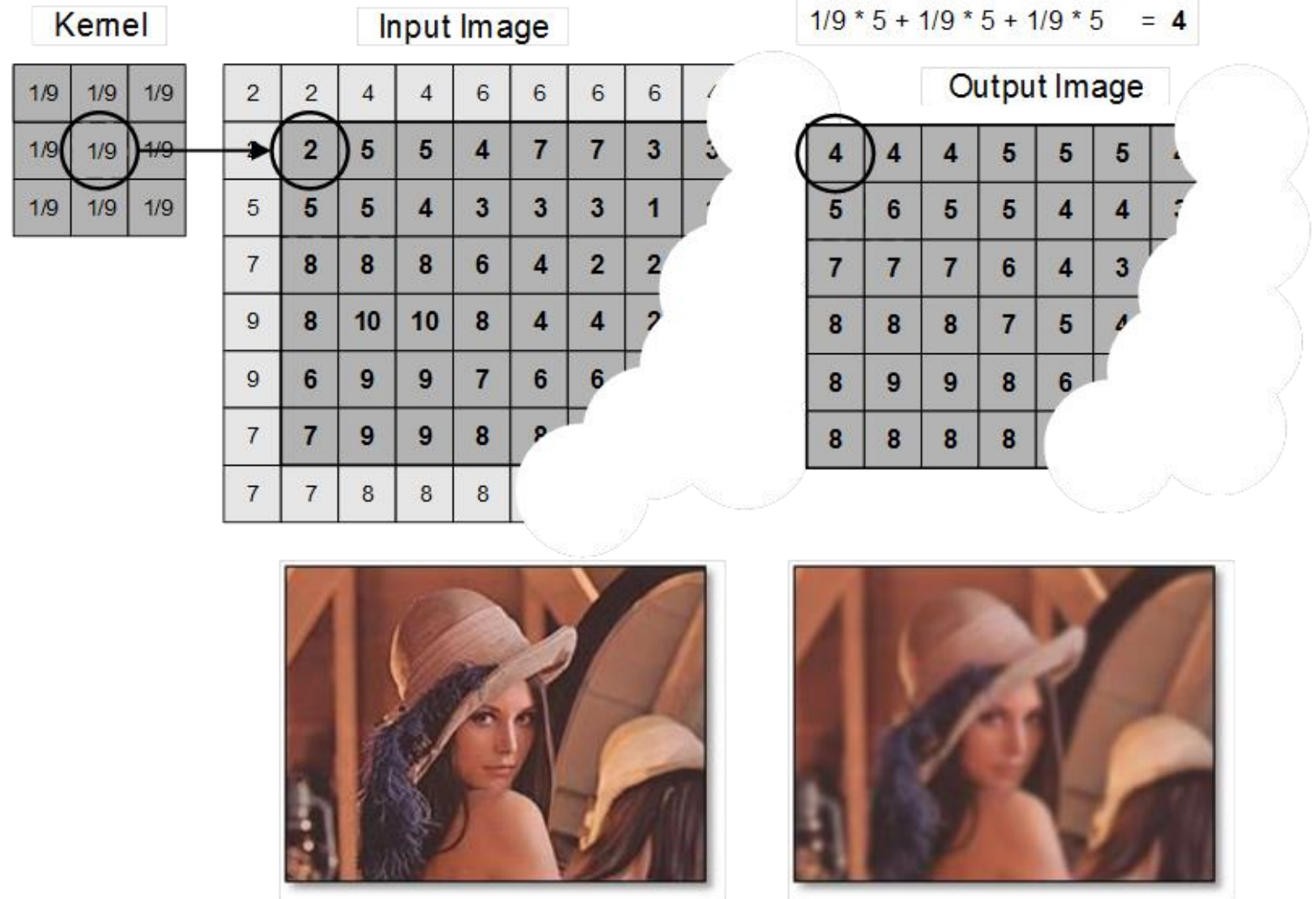


Image Filtering

```
image_gray = \
media.read_image('https://drive.google.com/uc?export=download&\
confirm=yes&id=1oG1LYW86HXWt0QoOuAF4j6NnX5KEHevC')

media.show_image(image_gray)
[height, width] = image_gray.shape

output = np.zeros(image_gray.shape, dtype='uint8')
winsize = 21
half_win = winsize//2 # window is 3x3, starts from i-1 to i+1 (included)

# loop works for any odd-sized square filter
for i in range(half_win, height-half_win):
    for j in range(half_win, width-half_win):
        window = image_gray[i-half_win:i+half_win+1, j-half_win:j+half_win+1]
        output[i,j] = np.sum(window.flatten())/(winsize*winsize)

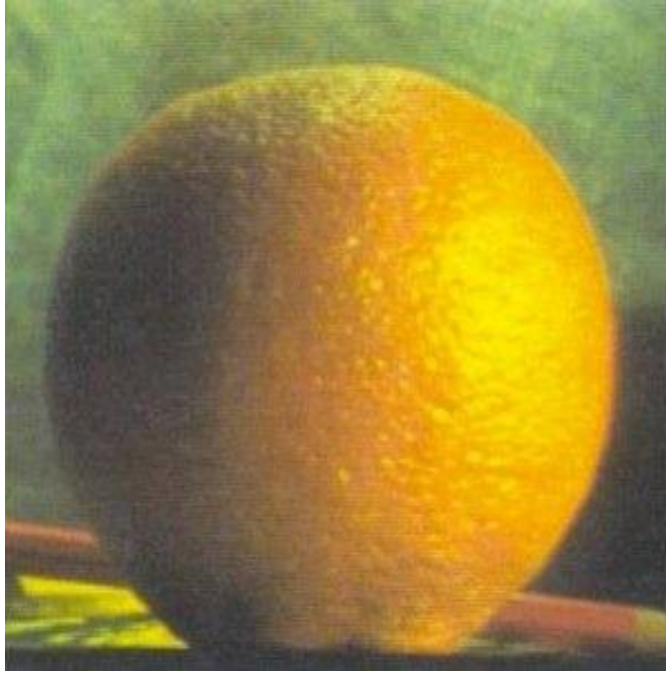
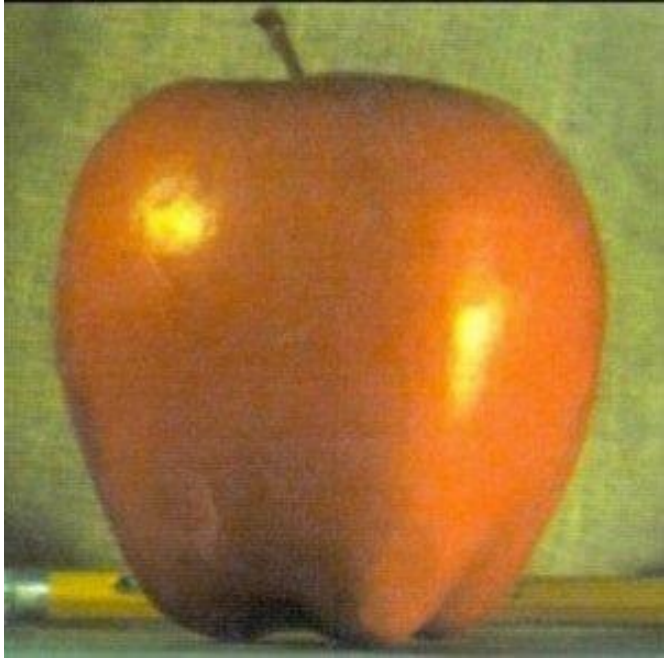
media.show_image(output)
```

TODO: more Filters

Make window LARGER to see effects

Try median filter

Image Morphing



$$0.5 * \text{apple} + 0.5 * \text{orange} = \text{orapple}$$

TODO: Image Morphing

```
#image morphing
```

```
apple = \
```

```
media.read_image('https://drive.google.com/uc?export=download&\nconfirm=yes&id=19GafyZCSMpR5FhCmkhXGZZuw4LklsLXP')
```

```
orange = \
```

```
media.read_image('https://drive.google.com/uc?export=download&\nconfirm=yes&id=1XCK8YPB9Pss63ucYoDbYpUQplScMitWC')
```

```
blend = ?????
```

```
blend=blend.astype('uint8')
```

```
media.show_image(blend)
```

[Files on Google Drive]

1. Find the ID of the file in the shareable link:

`https://drive.google.com/file/d/ID/view?usp=sharing`

2. Use the aforementioned ID as follows:

`https://drive.google.com/uc?export=download&confirm=yes&id=ID`